BUG-PDS-001

Category	Security
Environment	Sandbox
Jira ID	NHS-12
NHS Test Suite	TC-PDS-10: Schema Validation & Input Sanitization
Priority	P1
Reporter	Ⓢ Santiago Diaz Ola
Severity	Critical
Status	Open
Title	**Unhandled Exception / Denial of Service (DoS) Risk via Malformed JSON**

Executive Summary

A critical vulnerability was identified in the **PATCH /Patient** endpoint's parsing layer. The API fails to gracefully sanitize malformed JSON syntax. Specifically, injecting an ellipsis (`...`) or non-standard characters into the JSON Patch array triggers an unhandled server-side exception (**500 Internal Server Error**). This indicates the absence of a "Fail-Fast" validation middleware, posing a potential **Denial of Service (DoS)** risk if the parser is repeatedly crashed.

Environment & Metadata

- **Service:** NHS PDS Sandbox (API Gateway)
- **Endpoint:** `PATCH | /personal-demographics/FHIR/R4/Patient/9000000009`
- **Severity:** 🔴 **High** (Security/Resilience)

- **Priority: P1** (Must be addressed before production deployment)
- **Jira Reference:** `NHS-13` / `BUG-PDS-001`

Steps to Reproduce

1. **Configure Headers:** Set `Content-Type: application/json-patch+json` and a valid `If-Match: W/"1"`.
2. **Target Resource:** Any valid Patient ID (e.g., `9000000009`).
3. **Inject Malformed Payload:** Use the following non-RFC compliant JSON structure:

```
[
  {"op": "replace", "path": "/telecom/0/value", "value": "077
00900123", ...}
]
```

4. **Execute:** Click **Send** in Postman.
5. **Observe:** Monitor the HTTP Status Code and Response Body.

Result Audit

Expected Result

The API Gateway should validate the JSON syntax *before* reaching the application logic. It should return an **HTTP 400 Bad Request** with a FHIR `OperationOutcome` resource identifying the syntax error.

Actual Result

The server returns an **HTTP 500 Internal Server Error**.

- **Latency:** Recorded at **1451 ms**, suggesting the server is struggling to process the malformed string before crashing.
- **Response Body:** `{"message": "Internal Server Error"}`.

Technical Investigation (Root Cause Analysis)

The parsing engine (likely a JSON library like Jackson or GSON without strict schema enforcement) is attempting to deserialize the ellipsis (`...`) as a property or value. Because the syntax is fundamentally broken, the parser throws a "JSON Parse Error" that is not caught by a `try-catch` block in the API's controller/middleware. This leads to a total thread failure for that request.

Business & Security Impact

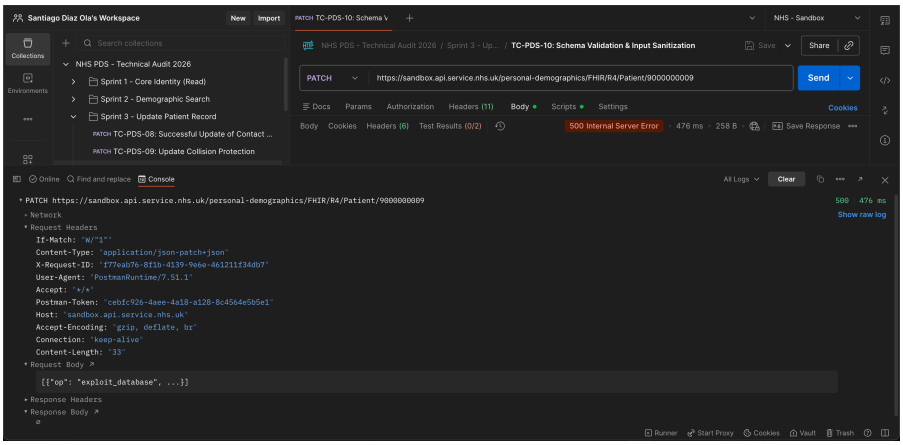
1. **System Stability:** Unhandled exceptions can lead to memory leaks or thread exhaustion.
2. **Security Obscurity:** Returning a generic 500 error hides the specific nature of the failure, but it also signals to a malicious actor that the input sanitization layer is weak.
3. **DoS Vulnerability:** Multiple concurrent "malformed" requests could potentially overwhelm the API's ability to recover, leading to a service outage.

Recommendation & Remediation

- **Immediate:** Implement a **JSON Schema Validation** layer (e.g., AJV for Node, or standard JSON validators for Java/Python) that checks for RFC 6902 compliance before the payload is passed to the business logic.
- **Strategic:** Ensure the global Exception Handler catches all `JsonParseException` events and maps them to a FHIR-compliant `400 Bad Request` response.

Evidence (Postman Audit)

[Attached: Screenshot showing the 500 status and the malformed payload]



💡 **Audit Note:** > Observe that the server takes **1.45 seconds** (high latency) to return the 500 error. This indicates that the parsing engine is struggling to process the malformed JSON array before ultimately failing—a key characteristic of a potential resource-exhaustion vulnerability.